RDST.jl — Streamable Random Number Generators

RDST.jl contributors

Contents

RDST.jl Documentation

Streamable pseudo-random number generators for Julia: MRG32k3a (L'Ecuyer) and xoshiro256+ (Blackman & Vigna), with non-overlapping streams and substreams.

Contents

Quick example

```julia
using RDST

gen  = MRG32k3aGen()
rngA = next_stream!(gen)      # independent stream A
rngB = next_stream!(gen)      # independent stream B

rand(rngA)                   # Float64 in [0, 1)
next_substream!(rngA)        # move to the next substream of A
reset_stream!(rngA)          # rewind stream A to its beginning
```

Getting Started

RDST.jl provides pseudo-random number generators with support for **non-overlapping streams and substreams**, following the stream/substream model of L'Ecuyer et al. (2002).

Installation

```julia
using Pkg
Pkg.add(url = "https://github.com/wirelessroom2/RDST.jl")
```

or, from a local clone:

```
using Pkg
Pkg.develop(path = "path/to/RDST.jl")
```

Requirements: Julia ≥ 1.6. The only dependency is the standard library `Random`.

Choosing a generator

RDST.jl implements the full xoshiro/xoroshiro family of Blackman & Vigna plus L'Ecuyer's MRG32k3a. All variants of a xoshiro family share the same linear transition and jump constants; only the output scrambler differs.

Type	Family	State	Period	Scrambler
`Xoroshiro128p`	xoroshiro128	128 bits	2^128 − 1	`s0 + s1`
`Xoroshiro128ss`	xoroshiro128	128 bits	2^128 − 1	high bits, **
`Xoroshiro128pp`	xoroshiro128	128 bits	2^128 − 1	high bits, ++
`Xoshiro256p`	xoshiro256	256 bits	2^256 − 1	`s0 + s3`
`Xoshiro256ss`	xoshiro256	256 bits	2^256 − 1	high bits, **
`Xoshiro256pp`	xoshiro256	256 bits	2^256 − 1	high bits, ++
`Xoshiro512p`	xoshiro512	512 bits	2^512 − 1	`s0 + s2`
`Xoshiro512ss`	xoshiro512	512 bits	2^512 − 1	high bits, **
`Xoshiro512pp`	xoshiro512	512 bits	2^512 − 1	high bits, ++
`MRG32k3a`	combined MRG (L'Ecuyer)	192 bits	≈ 2^191	native `Float64`

See Generator Comparison for guidance and benchmarks.

First numbers

```
using RDST

rng = MRG32k3a()      # default seed [12345, ..., 12345]
rand(rng)             # 0.12701112204657714
rand(rng)             # 0.3185275653967945

x = Xoshiro256p(UInt64[1, 2, 3, 4])
rand(x)               # Float64 in [0, 1)
```

Supported output types

MRG32k3a

- `Float64` (native), plus `Float32`, `Float16` via conversion
- `UInt8`, `UInt16`, `UInt32`, `UInt64`, `UInt128`
- `Int8`, `Int16`, `Int32`, `Int64`, `Int128` (same bit patterns)
- `Bool`

- Ranges: `rand(rng, 1:10)` works through the standard `Random` machinery

Note: integer outputs of MRG32k3a are built from its native ~31-bit resolution; they are not uniform over their full width. Use them for indices, choices and flags rather than cryptography.

Xoshiro256p

- `Float64` (native path), `Float32`, `Float16`
- `UInt64`
- Ranges: `rand(rng, r::UnitRange{Int64})`

Reproducibility

Every generator accepts an explicit seed:

```
rng = MRG32k3a([42, 1, 2, 3, 4, 5])
x   = Xoshiro256p(UInt64[0xdead, 0xbeef, 0xcafe, 0xbabe])
```

Seeds are validated by `checkseed`: a valid MRG32k3a seed has 6 non-negative values; the first three must be < m1 and not all zero, the last three < m2 and not all zero.

The standard Julia interface also works with every generator:

```
Random.seed!(rng, 42)          # integer seed (splitmix64 expansion)
Random.seed!(rng, [7,7,7,8,8,8])   # explicit MRG32k3a seed (validated)
```

Full Random API

RDST generators are drop-in substitutes for Julia's built-in RNGs: anywhere a function accepts an `AbstractRNG`, you can pass an RDST generator and keep your stream/substream control.

```
using RDST, Random

rng = next_stream!(MRG32k3aGen())     # any RDST generator works here

rand(rng, 5)                          # Vector{Float64}, 5 draws
A = rand(rng, Float64, 2, 3)          # 2x3 matrix
z = randn(rng)                        # standard normal
e = randexp(rng)                      # exponential
v = shuffle(rng, collect(1:8))        # shuffled copy
p = randperm(rng, 6)                  # random permutation
buf = Vector{Float64}(undef, 3)
rand!(rng, buf)                       # fill in place
Random.randstring(rng, 10)            # random string
```

Seeding through the standard interface makes results reproducible:

```
Random.seed!(rng, 42)
u1 = [rand(rng) for _ in 1:3]
Random.seed!(rng, 42)
```

```julia
u2 = [rand(rng) for _ in 1:3]
u1 == u2                                  # true
```

Only `sample` requires StatsBase.jl and is out of scope.

Next steps

- Streams & substreams
- API reference
- Implementation notes

Streams & Substreams

The central abstraction of RDST.jl is the *stream*: a long, non-overlapping subsequence of a generator's period. Streams can themselves be split into *substreams*. This is the architecture recommended by L'Ecuyer et al. (2002) for parallel and replicated stochastic simulation:

- **Streams** are handed to independent replications or parallel workers; their outputs never overlap, so replications are statistically independent.
- **Substreams** delimit scenarios *within* one replication; rewinding a substream while keeping the scenario structure enables techniques such as common random numbers.

The two object families

1. **Stream generators** (`<: AbstractRNGStream`) hold the seed of the *next* stream to be produced:
   - `MRG32k3aGen`
   - `Xoshiro256plusGen`
2. **RNGs** (`<: AbstractStreamableRNG`) actually produce numbers and can move along streams/substreams:
   - `MRG32k3a`
   - `Xoshiro256p`

Producing independent streams

```julia
using RDST

gen = MRG32k3aGen()              # or Xoshiro256plusGen(UInt64[...])

worker1 = next_stream!(gen)        # stream 1
worker2 = next_stream!(gen)        # stream 2 — guaranteed disjoint from stream 1
worker3 = next_stream!(gen)        # stream 3 — ...
```

Each call to `next_stream!` advances the generator's internal seed by a huge leap (2^127 values for MRG32k3a, a full `long_jump!` for Xoshiro256+), which is what guarantees non-overlap.

```julia
# Multithreaded simulation: one stream per thread.
# IMPORTANT: the generator object itself is not thread-safe — never share it;
# ship each worker its own stream *starting seed* instead.
using RDST, Base.Threads

gen = MRG32k3aGen()
starts = Vector{Vector{Int}}(undef, nthreads())
for t in 1:nthreads()
    starts[t] = copy(get_state(gen))   # seed of stream t
    next_stream!(gen)                  # advance to the following stream
end

results = Vector{Float64}(undef, nthreads())
@threads for t in 1:nthreads()
    s = starts[t]
    rng = MRG32k3a(s, s, s)            # rebuild stream t on this thread
    results[t] = sum(rand(rng) for _ in 1:10^5)
end
```

The same pattern works with Distributed: compute the list of starting seeds on the master process and send one entry per worker (`@spawnat` / `remotecall`), then rebuild the generator locally with the triple constructor.

Navigating substreams

Within a single RNG, three functions move along the stream/substream hierarchy (all return the modified RNG):

Function	Effect
`reset_stream!(rng)`	jump to the very beginning of the current stream
`reset_substream!(rng)`	jump to the beginning of the current substream
`next_substream!(rng)`	advance to the next substream

Example — common random numbers across two scenarios:

```julia
rng = next_stream!(MRG32k3aGen())

rand(rng)                 # consume some variates of substream 1...
rand(rng)

next_substream!(rng)      # switch to substream 2 for scenario B
uB = rand(rng)

reset_stream!(rng)        # replay substream 1 from scratch for scenario A
uA = rand(rng)            # same underlying uniforms as the first draw
```

For Xoshiro256+, `next_substream!` is implemented as a `short_jump!` (2^96-value leap) and `reset_stream!`/`next_substream!` manipulate the saved `Bg`/`Ig` checkpoints.

Saving, restoring, jumping

```
state = get_state(rng)     # a copy; safe to store
# ... consume numbers ...
```

To restore an MRG32k3a state, rebuild the generator with the triple constructor (`Cg`, `Bg`, `Ig` all set to the snapshot):

```
snapshot = get_state(rng)
xs = [rand(rng) for _ in 1:5]
clone = MRG32k3a(snapshot, snapshot, snapshot)
rand(clone) == xs[1]       # true — resumes exactly after the snapshot
```

MRG32k3a additionally supports arbitrary jumps inside a stream:

```
advance_state!(rng, e, c)
```

moves the state forward by `n` steps, where `n = 2^e + c` (`e` may be negative, and negative `c` moves backwards). This costs O(log n) matrix operations, independent of the distance jumped.

Guarantees

- Streams produced by successive calls to `next_stream!` on the same generator object are **provably non-overlapping**.
- Substreams likewise partition a stream into disjoint blocks (length ≈ 2^76 steps for MRG32k3a).

API Reference

All names below are exported by the `RDST` module unless marked *(internal)*.

Types

`AbstractRNGStream` — supertype of stream *generators* (objects that mint new independent streams): `MRG32k3aGen`, `Xoshiro256plusGen`.

`AbstractStreamableRNG <: Random.AbstractRNG` — supertype of usable RNGs that can navigate streams and substreams: `MRG32k3a`, `Xoshiro256p`.

MRG32k3a

Constructors

Signature	Description
`MRG32k3a()`	default seed `[12345, 12345, 12345, 12345, 12345, 12345]`
`MRG32k3a(x::Vector{Int})`	seed all three states (`Cg = Bg = Ig = x`)
`MRG32k3a(x, y, z::Vector{Int})`	explicit current/substream/stream starts

Seeds are validated with `checkseed`; invalid seeds throw an `AssertionError`.

Functions

`rand(rng::MRG32k3a) -> Float64` Uniform in [0, 1) with ~32 bits of resolution. This is the native output.

`rand(rng::MRG32k3a, T)` Supported `T`: `Float64`, `Float32`, `Float16`, `UInt8…UInt128`, `Int8…Int128`, `Bool`, and any `UnitRange` (e.g. `rand(rng, 1:10)`).

`reset_stream!(rng) -> rng` Set the current state (`Cg`) *and* the substream start (`Bg`) back to the stream start (`Ig`).

`reset_substream!(rng) -> rng` Set `Cg` back to `Bg`.

`next_substream!(rng) -> rng` Advance `Bg` to the next substream boundary (jump of ≈ 2^76 steps) and set `Cg = Bg`.

`advance_state!(rng, e::Integer, c::Integer)` Jump forward/backward inside the current substream by `n` steps where `n = 2^e + c`. Negative `e` or `c` move backwards. Cost is O(max(|e|, log |c|)) matrix operations.

`get_state(rng) -> Vector{Int}` Return a copy of `Cg`.

`checkseed(x) -> Bool` `true` iff `x` has length 6, all entries ≥ 0, entries 1–3 are < m1 and not all zero, entries 4–6 are < m2 and not all zero.

`DEFAULT_SEED` The constant seed vector used by `MRG32k3a()` and `MRG32k3aGen()`.

`copy(m::MRG32k3a) -> MRG32k3a` Independent deep copy of the generator (all three state vectors).

Stream generation

Signature	Description
`MRG32k3aGen()`, `MRG32k3aGen(seed)`	create a stream generator
`next_stream!(gen) -> MRG32k3a`	return a new stream; internal seed leaps 2^127 values ahead
`get_state(gen) -> Vector{Int}`	seed that will be used by the next `next_stream!` call

Xoshiro256+

Constructors

Signature	Description
`Xoshiro256p(s::NTuple{4,UInt64})` / `Xoshiro256p(v::Vector{<:Unsigned})`	`Cg = Bg = Ig = s`
`Xoshiro256p(x, y, z)`	explicit current/substream/stream states

Functions

`rand(rng::Xoshiro256p) -> Float64` Uniform in [0, 1), built from a full 64-bit draw.

`rand(rng::Xoshiro256p, T)` Supported `T`: `Float64`, `Float32`, `Float16`, `UInt64`, and `UnitRange{Int64}` (uniform, unbiased modulo sampling).

`srand!(rng, seed::Vector{UInt64}) -> rng` Re-seed an existing generator (first 4 words are used).

`reset_stream!(rng) -> rng`, **`reset_substream!(rng) -> rng`**, **`next_substream!(rng) -> rng`** Same semantics as for MRG32k3a; `next_substream!` performs a `short_jump!` (leap of ≈ 2^96 values).

`short_jump!(rng) -> rng` Jump to the beginning of the next 2^96-value block (Vigna's short jump).

`long_jump!(rng) -> rng` Leap ≈ 2^192 values ahead — used to delimit streams.

`get_state(rng) -> Vector{UInt64}` Copy of the current state `Cg`.

`copy(rng) -> Xoshiro256p` Independent copy.

(internal) **`next(rng) -> UInt64`**: raw 64-bit output; also available as `rand(rng, UInt64)`.

Stream generation

Signature	Description
`Xoshiro256plusGen(seed::Vector{UInt64})`	create a stream generator (seed must have exactly 4 words)
`next_stream!(gen) -> Xoshiro256p`	new independent stream (`long_jump!` from the stored seed)
`srand!(gen, seed::Vector{UInt64}) -> gen`	reset the stored seed
`get_state(gen) -> Vector{UInt64}`	seed that will be used by the next `next_stream!` call

Xoshiro / xoroshiro families

All variants share one implementation (`LinRNG{N,S}` internally, `N` = state words, `S` = scrambler) and expose the same API as Xoshiro256+ above. Exported types:

RNGs	Stream generators
`Xoroshiro128p`, `Xoroshiro128ss`, `Xoroshiro128pp`	`Xoroshiro128pGen`, `Xoroshiro128ssGen`, `Xoroshiro128ppGen`
`Xoshiro256p`, `Xoshiro256ss`, `Xoshiro256pp`	`Xoshiro256plusGen`, `Xoshiro256ssGen`, `Xoshiro256ppGen`
`Xoshiro512p`, `Xoshiro512ss`, `Xoshiro512pp`	`Xoshiro512pGen`, `Xoshiro512ssGen`, `Xoshiro512ppGen`

Constructors accept an `NTuple{N,UInt64}` or a `Vector{<:Unsigned}` of length `N` (1 or 3 arguments). Available functions: `rand` (Float64/Float32/Float16, UInt64, ranges), `srand!`, `reset_stream!`, `reset_substream!`, `next_substream!`, `short_jump!`, `long_jump!`, `get_state`, `copy`.

`advance_state!(rng, e::Integer, c::Integer) -> rng` Jump forward by `n` steps (`n = 2^e + c` if `e > 0`, `n = -2^(-e) + c` if `e < 0`, `n = c` if `e = 0`) or backward if `n < 0`. Implemented by computing the jump polynomial `x^n mod p(x)` over GF(2) (p = the family's characteristic polynomial) and applying it to the current state; distances are reduced modulo the period. Only `Cg` moves — stream/substream boundaries are kept. Cost is O((64N)²) GF(2) operations (≈ 10–500 ms depending on family); for the standard distances prefer `short_jump!`/`long_jump!`, which use precomputed constants and are allocation-free.

Jump semantics (all xoshiro-family generators):

- `short_jump!(rng)` — jump anchored at the current **substream start** (`Bg`); lands at the next substream boundary (2^64 values for xoroshiro128, 2^128 for xoshiro256, 2^256 for xoshiro512).
- `long_jump!(rng)` — jump anchored at the **stream start** (`Ig`); delimits independent streams.
- `next_stream!(gen)` applies the long-jump polynomial to the stored seed.

All jump constants are the official ones from xoshiro.di.unimi.it and outputs are validated byte-for-byte against the original C implementations.

Method index

Function	MRG32k3a	Xoshiro256p
`rand` (Float64)	yes	yes
`rand` (other floats/ints/bool)	yes	partial
`rand` (range)	yes	yes (`UnitRange{Int64}`)
`reset_stream!`	yes	yes

Function	MRG32k3a	Xoshiro256p
`reset_substream!`	yes	yes
`next_substream!`	yes	yes
`advance_state!`	yes	—
`srand!`	—	yes
`get_state`	yes	yes
`next_stream!` (via Gen)	yes	yes

Implementation Notes

MRG32k3a

MRG32k3a is L'Ecuyer's combined multiple recursive generator of order 3. It maintains two 3-component integer states updated by the recurrences

```
p1 = (1403580 · x2 − 810728 · x1) mod m1,      m1 = 2^32 − 209
p2 = (527612  · y2 − 1370589 · y0) mod m2,    m2 = 2^32 − 22853
u  = (p1 − p2) / m1   (with wrap-around) ∈ [0, 1)
```

The combined generator has period ≈ 2^191.

Modular arithmetic without overflow

All state components stay below `m1 < 2^32`, so products fit in `Float64` exactly (< 2^53). The helper `MultModM(a, s, c, m)` computes `(a·s + c) mod m` using this fact and falls back to a split-by-2^17 computation when intermediate values would exceed 2^53. Matrix helpers:

- `MatVecModM(A, s, m)` — matrix–vector product mod m
- `MatMatModM(A, B, m)` — matrix product mod m
- `MatTwoPowModM(A, e, m)` — A raised to the power 2^e
- `MatPowModM(A, n, m)` — A^n by binary exponentiation

Streams, substreams, jumps

Moving a stream forward by k steps is a linear operation on its state, hence a precomputed matrix power applied to the state vector:

Operation	Matrix	Step
`next_stream!(gen)`	`A1p127`, `A2p127`	2^127 values
`next_substream!(rng)`	`A1p76`, `A2p76`	2^76 values
`advance_state!(rng, e, c)`	`MatTwoPowModM` / `InvA1`, `InvA2`	arbitrary (inverse matrices allow backward jumps)

The RNG keeps three checkpoints: `Cg` (current), `Bg` (substream start), `Ig` (stream start), which is what makes `reset_substream!` and `reset_stream!` O(1).

Xoshiro256+

xoshiro256+ (Blackman & Vigna, 2019) keeps four 64-bit words and produces one word per step with only shifts, XORs and rotations:

```
result = s1 + s4
t = s2 << 17
s3 = xor(s3, s1);  s4 = xor(s4, s2);  s2 = xor(s2, s3);  s1 = xor(s1, s4);  s3 = xor(s3
```

The output word is the sum of two internal words (+ variant): fastest of the xoshiro family for float generation, though the low three bits of the raw output have limited linear complexity — irrelevant once scaled to `Float64` (53 bits used). Period: 2^256 − 1.

Jumps

`short_jump!` and `long_jump!` use Vigna's published jump polynomials. They are computed as XOR-linear combinations of states visited while stepping through the 256 bits of each jump constant. RDST.jl hoists the jump constants into compile-time tuples and evaluates jumps allocation-free on immutable state tuples.

RDST semantics differ subtly from the reference C code: jumps are **anchored** at stream boundaries. `short_jump!(rng)` first resets `Cg ← Bg`, then applies the jump polynomial, and stores the result in both `Cg` and `Bg`; `long_jump!(rng)` does the same with respect to `Ig`. This matches L'Ecuyer's stream model (`next_substream!`/`reset_stream!`) and makes scenario replay reproducible regardless of consumption inside the current substream.

Families and scramblers

The package implements all 64-bit variants from xoshiro.di.unimi.it through a single generic type `LinRNG{N,S}` (state words × scrambler):

- transition `_lin_step` for `NTuple{2}` (xoroshiro128: a=24, b=16, c=37), `NTuple{4}` (xoshiro256) and `NTuple{8}` (xoshiro512);
- scramblers `+`, `**` (`rotl(s2·5,7)·9`) and `++` (`rotl(s1+s4,23)+s1` for 256; family-specific constants);
- per-family jump constants shared by all scramblers.

Exported aliases: `Xoroshiro128p/ss/pp`, `Xoshiro256p/ss/pp`, `Xoshiro512p/ss/pp` plus matching `*Gen` stream generators.

All nine variants are regression-tested against byte-exact outputs produced by the original C implementations compiled with gcc (sequences, short jumps and long jumps).

Arbitrary forward/backward jumps

Because the transition is multiplication by x in GF(2)[x]/p(x) — with p the family's characteristic polynomial, as published in Vigna's reference files — one can jump any distance n in constant time by applying the polynomial `x^n mod p` through the

same accumulate-and-step loop as the fixed jumps. Backward distances use the multiplicative order of x: `x^(-n) = x^(2^deg-1-n)` (deg = 64N). RDST.jl implements a small GF(2) polynomial engine (`_poly_mul_mod`, `_poly_pow_x`, BigInt exponents reduced modulo the period) and exposes it as `advance_state!(rng, e, c)` with the exact distance convention of MRG32k3a. Correctness is property-tested: fixed jumps coincide with `short_jump!`, round trips (+k then -k) restore the state bit-for-bit, and backward-jumped generators re-emit previously seen values.

State representation

The state lives in `NTuple{4,UInt64}` fields. Immutable tuples let the JIT keep the whole working set in registers inside `next`, eliminating bounds checks and heap traffic that a `Vector{UInt64}` representation incurs. Measured throughput: $\approx 10^9$ draws/s on a single core.

Integer outputs of MRG32k3a

MRG32k3a natively yields ~31 bits per draw (the difference `p1 − p2`). Wider unsigned types are assembled from 16-bit chunks of that value; narrower types are truncations. Consequence: these integers are *not* uniform over their full width — fine for indexing/shuffling/flags in simulations, not for cryptographic use.

Testing strategy

Reference sequences were captured from the reference implementations (L'Ecuyer's C code semantics for MRG32k3a; Vigna's constants for the xoshiro jumps) and regression-checked after optimization: identical outputs are produced for every supported type, plus reset/jump/state round-trips.

Performance snapshot

Single core, Julia 1.12 (indicative):

Operation	Throughput
`rand(::MRG32k3a)`	$\approx 200 \times 10^6$/s
`rand(::Xoroshiro128p)`	$\approx 1200 \times 10^6$/s
`rand(::Xoshiro256p/ss/pp)`	$\approx 1340 \times 10^6$/s
`rand(::Xoshiro512p/ss/pp)`	$\approx 1150 \times 10^6$/s
`short_jump!` / `long_jump!`	0 allocations

Comparing generators: xoshiro variants vs MRG32k3a

RDST.jl lets you pick between two very different generator designs. This page helps you choose.

Side-by-side

Property	MRG32k3a	Xoroshiro128* (p/ss/pp)	Xoshiro256* (p/ss/pp)	Xoshiro512* (p/ss/pp)
State (bits)	192 (6 × Int64)	128	256	512
Period	≈ 2^191	2^128 − 1	2^256 − 1	2^512 − 1
Native output	`Float64` [0,1), ~32-bit resolution	`UInt64`	`UInt64`	`UInt64`
Throughput (this package, single core)	≈ 200 M/s	≈ 1200 M/s	≈ 1340 M/s	≈ 1150 M/s
Stream mechanism	matrix power A^2127 on the seed	`long_jump!` polynomial (2^96)	`long_jump!` (2^192)	`long_jump!` (2^384)
Substream mechanism	matrix power $A^2 76$	`short_jump!` (2^64)	`short_jump!` (2^128)	`short_jump!` (2^256)
Backward jumps	yes (`advance_state!`)	yes (`advance_state!`, GF(2) polynomials)	yes (`advance_state!`)	yes (`advance_state!`)
Arbitrary-jump cost	O(e) matrix products	O(deg²) GF(2) ops (~10–500 ms)	same	same
Equidistribution	well analysed theory	1-dim (**/++) / 0-dim (+)	3-dim (**/++) / 2-dim (+)	7-dim (**/++) / 6-dim (+)
BigCrush (Vigna's shootout)	passes	passes	passes	passes
Parallelism scale advised	large	mild (≈ 2^32 streams)	very large	extreme

(* means any of the +, **, ++ scramblers.)

When to choose MRG32k3a

- You need **provably non-overlapping streams with published, peer-reviewed parameters** (L'Ecuyer et al. 2002), e.g. to stay compatible with simulation libraries built on that model (SSJ, Simio, Arena-style stream ids…).
- Your application consumes **floats only**: MRG32k3a emits them natively.
- You can afford ~6x slower generation.

Note: backward jumping is *no longer* a differentiator — every RDST generator supports `advance_state!(rng, e, c)` with negative distances.

When to choose a xoshiro/xoroshiro variant

- You want **maximum throughput** (all are ≈ 6–7x faster than MRG32k3a here) or raw 64-bit integers.
- Choose your state size by parallelism needs:
  - `Xoroshiro128*`: embedded/GPU or few workers only — its 2^64 substreams per 2^32 streams are enough for mild parallelism. Note `Xoroshiro128p` has a mild Hamming-weight dependency after ~5 TB of output; prefer `ss`/`pp`.
  - `Xoshiro256*`: the sweet spot for general use (Julia itself uses xoshiro256++ as its default RNG). Use `+` if you generate *only* floating-point numbers from the high bits (~15% faster historically); otherwise prefer `++`/`**`, whose outputs are fully equidistributed.
  - `Xoshiro512*`: essentially never needed for period reasons — any period ≥ 2^256 is beyond every imaginable need — but handy when you want many more independent substreams than 2^128 without changing design.

Scrambler choice within a family

Scrambler	Output	Best for
`+`	sum of two state words	float-only workloads using the top bits; lowest 3 bits have low linear complexity
`**`	`rotl(s2·5, 7)·9`	all-purpose; maximal equidistribution (e.g. .NET/Lua default)
`++`	`rotl(s1+s4, k)+s1`	all-purpose; Rust's `SmallRng`, Julia's default

All three share the same transition and therefore the same jump constants: streams/substreams behave identically across scramblers of one family.

Practical notes

- Seeding: seed states must not be all-zero. For hash-like seeds, initialize with splitmix64 outputs (see Vigna's recommendation).
- The package's jumps are **anchored at stream boundaries** (`Cg ← Bg ← Ig`), matching the L'Ecuyer stream model: `next_substream!` always lands at the same place regardless of how far you consumed in the current substream, which makes scenario replay reproducible.

FAQ

Why is an all-zero state forbidden?

For the xoshiro/xoroshiro families the all-zero state maps to itself: the generator would output only zeros forever. MRG32k3a's two components have the same degeneracy

(an all-zero component stays zero). Constructors and `checkseed` reject such seeds; `Random.seed!(rng, ::Integer)` never produces one.

Are MRG32k3a integer outputs uniform over their full width?

No. MRG32k3a natively produces ~31 bits per draw (`p1 - p2`); wider integers are assembled from 16-bit chunks of that value. They are fine for indices, shuffling, flags and acceptance tests in simulations — not uniform over $2^{64/2}128$. Use a xoshiro variant when you need full-width raw words.

Why doesn't `sample(v, k)` work with RDST generators?

`sample` comes from StatsBase.jl, not from the Random standard library — even Julia's own RNGs do not provide it without that package. Everything in Random.jl works with RDST generators.

Where did `srand`, `next_stream`, `short_jump`, `long_jump` go?

They mutated their argument despite not ending in `!`; they are deprecated in favour of:

Deprecated	Replacement
`srand(rng/gen, seed)`	`srand!(rng/gen, seed)`
`next_stream(gen)`	`next_stream!(gen)`
`short_jump(rng)`	`short_jump!(rng)`
`long_jump(rng)`	`long_jump!(rng)`

The old names still work but emit a deprecation warning.

How do I run truly parallel simulations?

Never share one generator object across tasks/workers. Give each worker its own stream *starting seed* (see Streams & Substreams); streams produced by successive `next_stream!` calls are guaranteed non-overlapping.

Which generator should I pick?

See Generator Comparison. Short answer: `Xoshiro256pp` for general use, `MRG32k3a` for L'Ecuyer-compatible stream semantics.